

Riemann Curvature in $\mathcal{UD}$ + Compiler POC

Brian Beckman

Jan 2026

In this notebook, we derive the Riemann curvature tensor in the $\mathcal{UD}$ calculus from the commutator of covariant derivatives applied to a vector field A^μ in $\mathcal{UD}$. $\mathcal{UD}$ eliminates the tiresome, hazardous manual checking of indices and algebraic signs that every physicist has experienced. $\mathcal{UD}$ automates covariant differentiation, tensor quotienting, canonicalizing indices, etc.

In addition, this notebook demonstrates translating the $\mathcal{UD}$ expressions into calls of `MetricTensor` and `ChristoffelSymbol` from the Wolfram Function Repository (WFR). A proof-of-concept compiler backend performs this lowering. We plan to further develop this $\mathcal{UD}$ compiler in future versions to bridge the labor-saving benefits of $\mathcal{UD}$ to established tools in Wolfram and the WFR.

This notebook is an occasion for a major, non-breaking refactoring of $\mathcal{UD}$ over previous versions. The code still runs all the old tests and reproduces all the old results.

- A `.wl` block-version of code for this notebook, minus the prose and the compiler POC, is available at https://github.com/rebcabin/RelativityToolkit/blob/main/calculate_riemann.wl.

Initialize the Relativity Toolkit

Optional: Quit for a Fresh Kernel

- *Note, if you call `Quit` in a notebook, evaluation may hang. If you want to “Evaluate Notebook,” mark the following cell “Evaluatable” in the Cell→Property menu, call `Quit` once, then comment out the `Quit` cell, or mark it unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can “Evaluate Notebook” in a clean environment. Otherwise, you can evaluate cells in this notebook individually.*

```
(* Cell is not Evaluatable by default *)  
Quit[];
```

Download the Code

```
In[1]:= RTbranch = "main";
RTbase = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/" <>
RTbranch <> "/";
RTbust = "?t=" <> ToString[Round[AbsoluteTime[]]]; (*Force fresh download*)
RTrules = RTbase <> "RelativityToolkit.wl" <> RTbust;
RTtests = RTbase <> "RelativityToolkit_RegressionTests.wl" <> RTbust;
RTviz = RTbase <> "RelativityToolkit_VisualShowcase.wl" <> RTbust;
Get[RTrules];

» Relativity Toolkit version : 1.8.0
» Relativity Connection Symbol :  $\Gamma$ 
```

Optional: See all Tests and Results from Prior Works

Make any of the following cells evaluable if you wish. They're unevaluable by default to keep this notebook trim.

```
Get[RTtests];

In[*]:= Get[RTviz] (* no semicolon, on purpose *)
In[*]:= Get[RTbase <> "covariant_derivative.wl"]
In[*]:= Get[RTbase <> "law_of_gamma.wl"]
In[*]:= Get[RTbase <> "connections_to_christoffels.wl"]
```

Configure Relativity Connection Symbol

To properly compose covariant derivatives, $\mathcal{UD}$ must know the user's reserved symbols for non-tensorial connection coefficients. In gravitational physics, these force-producing connections are called Γ (capital gamma). Other theories like electrodynamics and Yang-Mills that can benefit from $\mathcal{UD}$ use other connection symbols.

We're going with Γ for this relativity/gravitation notebook.

```
In[8]:= SetConnection[ $\Gamma$ ];
```

Relativity Engine: Connection set to Γ

The Geometric Definition

Misner, Thorne, & Wheeler [MTW] start in Box 11.5, page 265, with a ***commutator of covariant derivatives***:

$$\mathcal{R}(\mathbf{n}, \mathbf{u}) = [\nabla_{\mathbf{n}}, \nabla_{\mathbf{u}}] - \nabla_{[\mathbf{n}, \mathbf{u}]} = [\nabla_{\mathbf{n}}, \nabla_{\mathbf{u}}].$$

$\uparrow$ $\left[\begin{array}{l} = 0 \text{ because } \mathbf{n} = (\partial/\partial n)_{\lambda} \text{ and} \\ \mathbf{u} = (\partial/\partial \lambda)_n \text{ commute} \end{array} \right]$

then they derive a component representation, Equation 11.12, in Exercise 11.3, page 276:

$$\begin{aligned}
 R^{\alpha}_{\beta\gamma\delta} &= \mathbf{Riemann}(\omega^{\alpha}, \mathbf{e}_{\beta}, \mathbf{e}_{\gamma}, \mathbf{e}_{\delta}) && \left[\begin{array}{l} \text{standard way to} \\ \text{calculate components} \end{array} \right] \\
 &= \langle \omega^{\alpha}, \mathcal{R}(\mathbf{e}_{\gamma}, \mathbf{e}_{\delta}) \mathbf{e}_{\beta} \rangle && [\text{by definition (11.9)}] \\
 &= \langle \omega^{\alpha}, (\nabla_{\gamma} \nabla_{\delta} - \nabla_{\delta} \nabla_{\gamma}) \mathbf{e}_{\beta} \rangle && \left[\begin{array}{l} \text{by definition (11.8) plus} \\ [\mathbf{e}_{\gamma}, \mathbf{e}_{\delta}] = 0 \text{ in coord. basis} \end{array} \right] \\
 &= \langle \omega^{\alpha}, \mathbf{e}_{\mu} \Gamma^{\mu}_{\beta\delta, \gamma} + (\mathbf{e}_{\nu} \Gamma^{\nu}_{\mu\gamma}) \Gamma^{\mu}_{\beta\delta} - \mathbf{e}_{\mu} \Gamma^{\mu}_{\beta\gamma, \delta} - (\mathbf{e}_{\nu} \Gamma^{\nu}_{\mu\delta}) \Gamma^{\mu}_{\beta\gamma} \rangle \\
 &= (\Gamma^{\mu}_{\beta\delta, \gamma} - \Gamma^{\mu}_{\beta\gamma, \delta}) \langle \omega^{\alpha}, \mathbf{e}_{\mu} \rangle + (\Gamma^{\nu}_{\mu\gamma} \Gamma^{\mu}_{\beta\delta} - \Gamma^{\nu}_{\mu\delta} \Gamma^{\mu}_{\beta\gamma}) \langle \omega^{\alpha}, \mathbf{e}_{\nu} \rangle, \\
 &\text{which reduces (upon using } \langle \omega^{\alpha}, \mathbf{e}_{\mu} \rangle = \delta^{\alpha}_{\mu} \text{) to (11.12).}
 \end{aligned}$$

The Derivation in Index Notation

MTW's geometric forms are too high-level for $\mathcal{UD}$. The index notation is just right. Let's start with the commutator

Step 1: Formulate and Differentiate

First, remind ourselves of the action of our covariant derivative, CD, trusted from earlier notebooks and tests. Apply CD to a concrete contravariant vector, A^{β} in lieu of MTW's abstract basis forms:

```
In[9]:= Echo[CD[A[U[α]], x[U[δ]]], "∇δAβ ≡ Aβ;δ ="];
```

» $\nabla_{\delta} A^{\beta} \equiv A^{\beta}_{;\delta} = A^{\alpha}_{\delta} + (A^{\lambda 11}) \Gamma^{\alpha}_{\delta \lambda 11}$

- Note the generated summation index. $\mathcal{UD}$ keeps track of such and prevents cross-capture with meaningful (free) indices in big calculations.
- TODO: we might add basis abstractions to a future version of $\mathcal{UD}$. It's lower priority, however, as we get the same results by application of Quotient Theorems.

The commutator applies another covariant derivative, as in $\nabla_{\gamma}(\nabla_{\delta} A^{\beta})$, then reverses the order of γ and δ and subtracts: $\nabla_{\gamma}(\nabla_{\delta} A^{\beta}) - \nabla_{\delta}(\nabla_{\gamma} A^{\beta})$. However, composing covariant derivatives blindly conceals a trap: the product rule of CD only pertains to products of tensors, but the inner CD produces products like $A.\Gamma$, a tensor times a non-tensor! The next section introduces the fix:

Boxing Non-Tensorial Terms

When we apply the “outer” ∇_{γ} to the “inner” $\nabla_{\delta} A^{\beta}$, CD must treat the “inner” $\nabla_{\delta} A^{\beta} = A^{\beta}_{;\delta} - A^{\lambda} \Gamma^{\beta}_{\delta \lambda}$ as a whole tensor. That means CD must not apply the ∇ product rule to the term $A^{\lambda} \Gamma^{\beta}_{\delta \lambda}$.

$\nabla(A.\Gamma)$ does not equal $(\nabla A).\Gamma + A.(\nabla \Gamma)$.

$\nabla \Gamma$ is nonsense! Γ is not a tensor, as we know from my earlier notebooks! Reserving Γ , globally, as a non-tensorial symbol for CD is the reason for calling `SetConnection` above.

While Γ is not a tensor, $A.\Gamma$ is. A new version of CD recognizes the “poison-pill” Γ in $A.\Gamma$ and bypasses the ∇ product rule. CD sends $A.\Gamma$ in an inert box down the line to a later *appropriate* application of the ordinary product rule from calculus. CD “glues the A to the Γ ” (look for “gamma glue” in the code: `RelativityToolkit.wl`, Rule 0 of CD).

With that realization, we can confidently set up $\nabla_Y(\nabla_\delta A^\beta)$ and $\nabla_\delta(\nabla_Y A^\beta)$ and visually spot the boxed $A.\Gamma$ terms at-a-glance:

```
In[10]:= Echo[term1 = CD[CD[A[U[α]], x[U[δ]]], x[U[γ]]], "∇γ(∇δAβ) ="];
Echo[term2 = CD[CD[A[U[α]], x[U[γ]]], x[U[δ]]], "∇δ(∇γAβ) ="];
```

$$\begin{aligned} \gg \nabla_Y(\nabla_\delta A^\beta) &= \frac{\partial^2 A^\alpha}{\partial x^\delta \partial x^\gamma} + \left(A^{\lambda 12} \right) \Gamma_{\delta \lambda 12}^\alpha, \gamma + \\ &\quad \left(A^{\lambda 13}, \delta \right) \Gamma_{\gamma \lambda 13}^\alpha - \left(A^{\alpha, \lambda 14} \right) \Gamma_{\delta \gamma}^{\lambda 14} + \left(A^{\lambda 12} \right) \Gamma_{\gamma \lambda 15}^\alpha \Gamma_{\delta \lambda 12}^{\lambda 15} - \left(A^{\lambda 12} \right) \Gamma_{\lambda 16 \lambda 12}^\alpha \Gamma_{\delta \gamma}^{\lambda 16} \\ \gg \nabla_\delta(\nabla_Y A^\beta) &= \frac{\partial^2 A^\alpha}{\partial x^\delta \partial x^\gamma} + \left(A^{\lambda 17} \right) \Gamma_{\gamma \lambda 17}^\alpha, \delta + \\ &\quad \left(A^{\lambda 18}, \gamma \right) \Gamma_{\delta \lambda 18}^\alpha - \left(A^{\alpha, \lambda 19} \right) \Gamma_{\gamma \delta}^{\lambda 19} + \left(A^{\lambda 17} \right) \Gamma_{\delta \lambda 20}^\alpha \Gamma_{\gamma \lambda 17}^{\lambda 20} - \left(A^{\lambda 17} \right) \Gamma_{\lambda 21 \lambda 17}^\alpha \Gamma_{\gamma \delta}^{\lambda 21} \end{aligned}$$

- *At the risk of overselling, no one should do this kind of calculation by hand. Ah, memories of lonely nights in the Physics Library with pen and paper! That's the whole point of $\mathcal{UD}$: delegating risky and tedious calculations to Wolfram.*

Look at the RAW commutator, with boxed terms, now:

```
In[12]:= Echo[rawCommutator = term1 - term2, "∇γ(∇δAβ) - ∇δ(∇γAβ) ="];
```

$$\begin{aligned} \gg \nabla_Y(\nabla_\delta A^\beta) - \nabla_\delta(\nabla_Y A^\beta) &= - \left(A^{\lambda 17} \right) \Gamma_{\gamma \lambda 17}^\alpha, \delta + \left(A^{\lambda 12} \right) \Gamma_{\delta \lambda 12}^\alpha, \gamma + \\ &\quad \left(A^{\lambda 13}, \delta \right) \Gamma_{\gamma \lambda 13}^\alpha - \left(A^{\lambda 18}, \gamma \right) \Gamma_{\delta \lambda 18}^\alpha - \left(A^{\alpha, \lambda 14} \right) \Gamma_{\delta \gamma}^{\lambda 14} + \left(A^{\lambda 12} \right) \Gamma_{\gamma \lambda 15}^\alpha \Gamma_{\delta \lambda 12}^{\lambda 15} - \\ &\quad \left(A^{\lambda 12} \right) \Gamma_{\lambda 16 \lambda 12}^\alpha \Gamma_{\delta \gamma}^{\lambda 16} + \left(A^{\alpha, \lambda 19} \right) \Gamma_{\gamma \delta}^{\lambda 19} - \left(A^{\lambda 17} \right) \Gamma_{\delta \lambda 20}^\alpha \Gamma_{\gamma \lambda 17}^{\lambda 20} + \left(A^{\lambda 17} \right) \Gamma_{\lambda 21 \lambda 17}^\alpha \Gamma_{\gamma \delta}^{\lambda 21} \end{aligned}$$

Now that all the explosive ∇ terms are evaluated, it's time to apply the product (Leibniz) rule from ordinary calculus via `ExpandDerivatives` and open these terms to ordinary Wolfram algebraic simplification.

Applying the Product Rule

```
In[13]:= Echo[expanded = ExpandDerivatives[rawCommutator], "expanded commutator :"];
```

$$\begin{aligned} \gg \text{expanded commutator} &: - \left(A^{\lambda 17} \right) \Gamma_{\gamma \lambda 17}^\alpha, \delta + \left(A^{\lambda 12} \right) \Gamma_{\delta \lambda 12}^\alpha, \gamma + \left(A^{\lambda 13}, \delta \right) \Gamma_{\gamma \lambda 13}^\alpha - \\ &\quad \left(A^{\lambda 17}, \delta \right) \Gamma_{\gamma \lambda 17}^\alpha + \left(A^{\lambda 12}, \gamma \right) \Gamma_{\delta \lambda 12}^\alpha - \left(A^{\lambda 18}, \gamma \right) \Gamma_{\delta \lambda 18}^\alpha - \left(A^{\alpha, \lambda 14} \right) \Gamma_{\delta \gamma}^{\lambda 14} + \left(A^{\lambda 12} \right) \Gamma_{\gamma \lambda 15}^\alpha \Gamma_{\delta \lambda 12}^{\lambda 15} - \\ &\quad \left(A^{\lambda 12} \right) \Gamma_{\lambda 16 \lambda 12}^\alpha \Gamma_{\delta \gamma}^{\lambda 16} + \left(A^{\alpha, \lambda 19} \right) \Gamma_{\gamma \delta}^{\lambda 19} - \left(A^{\lambda 17} \right) \Gamma_{\delta \lambda 20}^\alpha \Gamma_{\gamma \lambda 17}^{\lambda 20} + \left(A^{\lambda 17} \right) \Gamma_{\lambda 21 \lambda 17}^\alpha \Gamma_{\gamma \delta}^{\lambda 21} \end{aligned}$$

Step 2: Canonicalize and Eliminate Torsion

Now, canonicalize indices and apply the torsion rules, which express *physical* symmetry of Γ in general

relativity.

- *Other applications of differential geometry may have non-zero torsion, but not gravitation.*

```
In[14]:= Echo[simplified = (CanonicalizeIndices[expanded] //. torsionRules),
  "canon - torsion :"];
```

» canon - torsion : $-(A^{i1}) \Gamma_{i1\gamma,\delta}^\alpha + (A^{i1}) \Gamma_{i1\delta,\gamma}^\alpha + (A^{i1}) \Gamma_{i1\delta}^{i2} \Gamma_{i2\gamma}^\alpha - (A^{i1}) \Gamma_{i1\gamma}^{i2} \Gamma_{i2\delta}^\alpha$

Step 3: Apply Quotient Theorem

Riemann is peeking out! To recover Riemann alone, quotient-out the tensor, A (it's arbitrary), and we have a raw Riemann tensor.

- *To borrow jargon from functional programming, Riemann.A is “partially evaluated” on A . The quotient theorem undoes the partial evaluation by a kind of functional factoring that Haskell Curry formalized.*

```
In[15]:= Echo[rawRiemann1 = ExtractCoefficient[simplified, A], "raw Riemann :"];
```

» raw Riemann : $-\Gamma_{\gamma,\delta}^\alpha + \Gamma_{\delta,\gamma}^\alpha + \Gamma_{\delta}^{i2} \Gamma_{i2\gamma}^\alpha - \Gamma_{\gamma}^{i2} \Gamma_{i2\delta}^\alpha$

Step 4: Presentation

Produce readable Greek via $\mathcal{UD}$'s **TensorForm**. Bring our ground-truth image nearby for a final look. Except for ordering of terms and renaming of indices, (my $\lambda \rightarrow$ their β , my $\kappa \rightarrow$ their λ in the linear Γ terms, summation over my λ and their μ in the quadratic Γ terms), they're identical. $\mathcal{UD}$ has removed the hazard of getting the signs and indices wrong.

```
In[16]:= Echo[TensorForm[rawRiemann1], "R $^\alpha_{\lambda\gamma\delta}$  ="];
```

» $R^\alpha_{\lambda\gamma\delta} = -\Gamma^\alpha_{\lambda\gamma,\delta} + \Gamma^\alpha_{\lambda\delta,\gamma} - \Gamma^\alpha_{\kappa\delta} \Gamma^\kappa_{\lambda\gamma} + \Gamma^\alpha_{\kappa\gamma} \Gamma^\kappa_{\lambda\delta}$

$$= (\Gamma^\mu_{\beta\delta,\gamma} - \Gamma^\mu_{\beta\gamma,\delta}) \langle w^\alpha, e_\mu \rangle + (\Gamma^\nu_{\mu\gamma} \Gamma^\mu_{\beta\delta} - \Gamma^\nu_{\mu\delta} \Gamma^\mu_{\beta\gamma}) \langle w^\alpha, e_\nu \rangle,$$

```
In[58]:= TensorForm[rawRiemann1] /. {α → μ, λ → β, κ → λ}
```

Out[58]=

$$-\Gamma^\mu_{\beta\gamma,\delta} + \Gamma^\mu_{\beta\delta,\gamma} - \Gamma^\mu_{\lambda\delta} \Gamma^\lambda_{\beta\gamma} + \Gamma^\mu_{\lambda\gamma} \Gamma^\lambda_{\beta\delta}$$

Recap from GitHub Repo

You can also run a stored version of this calculation from the GitHub repo. The indices and order of terms are different by choice and/or accident. Cycling indices shows the results are mathematically identical to the above.

```
In[17]:= Get[RTbase <> "calculate_riemann.wl"]
```

$$\begin{aligned}
& \gg \nabla_\rho \nabla_\nu A^\mu: \frac{\partial^2 A^\mu}{\partial x^\rho \partial x^\nu} + \left(A^{\lambda 22} \right) \Gamma_{\nu \lambda 22}^\mu{}_{,\rho} - \left(A^{\mu, \lambda 24} \right) \Gamma_{\nu \rho}^{\lambda 24} - \\
& \quad \left(A^{\lambda 22} \right) \Gamma_{\nu \rho}^{\lambda 26} \Gamma_{\lambda 26 \lambda 22}^\mu + \left(A^{\lambda 23}{}_{,\nu} \right) \Gamma_{\rho \lambda 23}^\mu + \left(A^{\lambda 22} \right) \Gamma_{\nu \lambda 22}^{\lambda 25} \Gamma_{\rho \lambda 25}^\mu \\
& \gg \nabla_\nu \nabla_\rho A^\mu: \frac{\partial^2 A^\mu}{\partial x^\rho \partial x^\nu} + \left(A^{\lambda 27} \right) \Gamma_{\rho \lambda 27}^\mu{}_{,\nu} - \left(A^{\mu, \lambda 29} \right) \Gamma_{\rho \nu}^{\lambda 29} - \\
& \quad \left(A^{\lambda 27} \right) \Gamma_{\rho \nu}^{\lambda 31} \Gamma_{\lambda 31 \lambda 27}^\mu + \left(A^{\lambda 28}{}_{,\rho} \right) \Gamma_{\nu \lambda 28}^\mu + \left(A^{\lambda 27} \right) \Gamma_{\rho \lambda 27}^{\lambda 30} \Gamma_{\nu \lambda 30}^\mu \\
& \gg \text{raw commutator} \left(\left(A^{\lambda 22} \right) \Gamma_{\nu \lambda 22}^\mu{}_{,\rho} - \left(A^{\lambda 27} \right) \Gamma_{\rho \lambda 27}^\mu{}_{,\nu} - \right. \\
& \quad \left(A^{\mu, \lambda 24} \right) \Gamma_{\nu \rho}^{\lambda 24} + \left(A^{\mu, \lambda 29} \right) \Gamma_{\rho \nu}^{\lambda 29} - \left(A^{\lambda 22} \right) \Gamma_{\nu \rho}^{\lambda 26} \Gamma_{\lambda 26 \lambda 22}^\mu + \left(A^{\lambda 27} \right) \Gamma_{\rho \nu}^{\lambda 31} \Gamma_{\lambda 31 \lambda 27}^\mu - \\
& \quad \left(A^{\lambda 28}{}_{,\rho} \right) \Gamma_{\nu \lambda 28}^\mu - \left(A^{\lambda 27} \right) \Gamma_{\rho \lambda 27}^{\lambda 30} \Gamma_{\nu \lambda 30}^\mu + \left(A^{\lambda 23}{}_{,\nu} \right) \Gamma_{\rho \lambda 23}^\mu + \left(A^{\lambda 22} \right) \Gamma_{\nu \lambda 22}^{\lambda 25} \Gamma_{\rho \lambda 25}^\mu \\
& \gg \text{xpd commutator:} \left(A^{\lambda 22} \right) \Gamma_{\nu \lambda 22}^\mu{}_{,\rho} - \left(A^{\lambda 27} \right) \Gamma_{\rho \lambda 27}^\mu{}_{,\nu} - \left(A^{\mu, \lambda 24} \right) \Gamma_{\nu \rho}^{\lambda 24} + \left(A^{\mu, \lambda 29} \right) \Gamma_{\rho \nu}^{\lambda 29} - \\
& \quad \left(A^{\lambda 22} \right) \Gamma_{\nu \rho}^{\lambda 26} \Gamma_{\lambda 26 \lambda 22}^\mu + \left(A^{\lambda 27} \right) \Gamma_{\rho \nu}^{\lambda 31} \Gamma_{\lambda 31 \lambda 27}^\mu + \left(A^{\lambda 22}{}_{,\rho} \right) \Gamma_{\nu \lambda 22}^\mu - \left(A^{\lambda 28}{}_{,\rho} \right) \Gamma_{\nu \lambda 28}^\mu - \\
& \quad \left(A^{\lambda 27} \right) \Gamma_{\rho \lambda 27}^{\lambda 30} \Gamma_{\nu \lambda 30}^\mu + \left(A^{\lambda 23}{}_{,\nu} \right) \Gamma_{\rho \lambda 23}^\mu + \left(A^{\lambda 22} \right) \Gamma_{\nu \lambda 22}^{\lambda 25} \Gamma_{\rho \lambda 25}^\mu - \left(A^{\lambda 27}{}_{,\nu} \right) \Gamma_{\rho \lambda 27}^\mu \\
& \gg \text{Expanded – SealedBox diagnostic:} \left(A^{\lambda 22} \right) \Gamma_{\nu \lambda 22}^\mu{}_{,\rho} - \left(A^{\lambda 22} \right) \Gamma_{\nu \lambda 22}^\mu{}_{,\rho} - \\
& \quad \left(A^{\lambda 27} \right) \Gamma_{\rho \lambda 27}^\mu{}_{,\nu} + \left(A^{\lambda 27} \right) \Gamma_{\rho \lambda 27}^\mu{}_{,\nu} + \left(A^{\lambda 22}{}_{,\rho} \right) \Gamma_{\nu \lambda 22}^\mu - \left(A^{\lambda 27}{}_{,\nu} \right) \Gamma_{\rho \lambda 27}^\mu \\
& \gg \text{alg. xp + torsion elmin:} \left(A^{\lambda 11} \right) \Gamma_{11 \nu, \rho}^\mu - \left(A^{\lambda 11} \right) \Gamma_{11 \rho, \nu}^\mu - \left(A^{\lambda 11} \right) \Gamma_{11 \rho}^{i2} \Gamma_{i2 \nu}^\mu + \left(A^{\lambda 11} \right) \Gamma_{11 \nu}^{i2} \Gamma_{i2 \rho}^\mu \\
& \gg \text{test: has 2d derivs? False} \\
& \gg \text{raw Riemann} \Gamma_{\xi \nu, \rho}^\mu - \Gamma_{\xi \rho, \nu}^\mu - \Gamma_{\xi \rho}^{i2} \Gamma_{i2 \nu}^\mu + \Gamma_{\xi \nu}^{i2} \Gamma_{i2 \rho}^\mu
\end{aligned}$$

=== THE RIEMANN CURVATURE TENSOR ===

$$\gg R^\mu_{\lambda \nu \rho} = \Gamma^\mu_{\lambda \nu, \rho} - \Gamma^\mu_{\lambda \rho, \nu} - \Gamma^\kappa_{\lambda \rho} \Gamma^\mu_{\kappa \nu} + \Gamma^\kappa_{\lambda \nu} \Gamma^\mu_{\kappa \rho}$$

Proof: Notebook Equivalent to Script

```

In[59]:= TensorForm[CanonicalizeIndices[rawRiemann1] /. {α → μ, δ → ν, γ → ρ}]
TensorForm[CanonicalizeIndices[rawRiemann]]

Out[59]=
Γμλ ν, ρ - Γμλ ρ, ν - Γκλ ρ Γμκ ν + Γκλ ν Γμκ ρ

Out[60]=
Γμλ ν, ρ - Γμλ ρ, ν - Γκλ ρ Γμκ ν + Γκλ ν Γμκ ρ

```

Compiler Backend Proof of Concept

In this section of the notebook, we develop a single-purpose compiler component that lowers $\mathcal{UD}$ expressions of the Schwarzschild metric and its Christoffel symbols onto functions in the Wolfram Function Repository (WFR). As above, this section is presented in alternating prose-and-code style rather than in .wl block style.

Pull the Metric

Pull the covariant Schwarzschild metric from the WFR:

```
In[19]:= metricObj = ResourceFunction["MetricTensor"] ["Schwarzschild"]
```

```
Out[19]=
```

MetricTensor [ Type: Covariant Symbol: $g_{\mu\nu}$
Dimensions: 4 Signature: Indeterminate]

Rationalize Symbols

Inexplicably, the metric from WFR has strings containing formal symbols, which (annoyingly) print just like formal symbols. It took a bit of effort to debug this.

```
In[20]:= (rawCoords = metricObj["Coordinates"]) // FullForm
```

```
Out[20]//FullForm=
```

```
List["\[FormalT]", "\[FormalR]", "\[FormalTheta]", "\[FormalPhi]"]
```

Wolfram can't take derivatives with respect to strings. Just map them to ordinary formal variables, input via Esc-.t-Esc, etc.:

```
In[21]:= workingCoords = {t, r,  $\theta$ ,  $\phi$ };
simpleMap = Thread[rawCoords → workingCoords];
```

Covariant $g_{\mu\nu}$

```
In[23]:= gDD = metricObj["MatrixRepresentation"] /. simpleMap /. {"M" → M}
```

```
Out[23]=
```

$$\left\{ \left\{ -1 + \frac{2M}{r}, 0, 0, 0 \right\}, \left\{ 0, \frac{1}{1 - \frac{2M}{r}}, 0, 0 \right\}, \left\{ 0, 0, r^2, 0 \right\}, \left\{ 0, 0, 0, r^2 \sin^2[\theta] \right\} \right\}$$

Contravariant $g^{\mu\nu}$

We'll need the inverse, i.e., the contravariant metric, later for the Christoffel symbols

```
In[24]:= gUU = FullSimplify[Inverse[gDD]]
```

```
Out[24]=
```

$$\left\{ \left\{ -\frac{r}{r - 2M}, 0, 0, 0 \right\}, \left\{ 0, 1 - \frac{2M}{r}, 0, 0 \right\}, \left\{ 0, 0, \frac{1}{r^2}, 0 \right\}, \left\{ 0, 0, 0, \frac{\csc^2[\theta]}{r^2} \right\} \right\}$$

Compiling Derivatives of the Metric

Let's do a two-pass pipeline compilation of derivatives: algebrize then differentiate.

- Many areas of physics and mathematics employ 0-based indexing, while Wolfram is 1-based. We have found by experience that the best way to handle this discrepancy is to translate between indexing schemes at the last possible time and place: in the deepest Part expressions and in return values. Any other approach tends to proliferate the index translations into unwanted places.

```

In[25]:= ClearAll[DifferentiateTheMetric];
DifferentiateTheMetric[expr_, gDD_, gUU_] :=
Module[{lowerGeometry, activateCalculus},
  (* PASS 1: Geometry→Algebra*)
  (* Correct for physics g_00 to Wolfram [[1,1]] at the last possible moment *)
  lowerGeometry = expr //. {
    g[D[a_], D[b_]] => gDD[[1 + a, 1 + b]],
    g[U[a_], U[b_]] => gUU[[1 + a, 1 + b]]};
  (* PASS 2: transpile UD Partial → Wolfram D *)
  activateCalculus = lowerGeometry //.
    {Partials[content_, x[U[dir_]]] =>
      D[content, workingCoords[[1 + dir]]]};
  Simplify[activateCalculus]];

```

Smoke test

Notice the conventional 0-based indexing from physics and mathematics:

```

In[27]:= - 2 M / r^2 === DifferentiateTheMetric[Partials[g[D[0], D[0]], x[U[1]]], gDD, gUU]
Out[27]= True

```

Construct Γ Components in $\mathcal{UD}$

Let's carefully construct components of Γ in terms of components of g in $\mathcal{UD}$, again with conventional (non-Wolfram) zero-based indexing. Recalling from an earlier notebook of mine:

$$\Gamma_{\mu\nu}^{\sigma} = \frac{1}{2} \left(g^{\sigma\lambda} \left(g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda} \right) \right)$$

```

In[28]:= symbolicGamma[σ_, μ_, ν_] :=
Module[{λ}, (* dummy index *)
  Sum[1/2 g[U[σ], U[λ]] *
    (Partials[g[D[λ], D[μ]], x[U[ν]]] +
     Partials[g[D[λ], D[ν]], x[U[μ]]] -
     Partials[g[D[μ], D[ν]], x[U[λ]]]),
    {λ, 0, 3}]];

```

Let's differentiate the 64 internal terms of the Schwarzschild metric one at a time. Here is the resulting array that we'll compare against the "ground truth" `ChristoffelSymbol` from WFR:

```
In[29]:= (myGammaArray = Table[
  DifferentiateTheMetric[
    symbolicGamma[l, m, n], gDD, gUU],
  {l, 0, 3}, {m, 0, 3}, {n, 0, 3}]) // MatrixForm
```

Out[29]//MatrixForm=

$$\begin{pmatrix} \begin{pmatrix} 0 \\ \frac{M}{r^2-2rM} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} \frac{M}{r^2-2rM} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ \begin{pmatrix} \frac{(r-2M)M}{r^3} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ -\frac{M}{r^2-2rM} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ -r+2M \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -(r-2M)\sin[\theta]^2 \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ \frac{1}{r} \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -\cos[\theta]\sin[\theta] \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{r} \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \cot[\theta] \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ \cot[\theta] \\ 0 \end{pmatrix} \end{pmatrix}$$

Ground Truth

```
In[30]:= (truthArray = Simplify[
  ResourceFunction["ChristoffelSymbol"][gDD, workingCoords]]) // MatrixForm
```

Out[30]//MatrixForm=

$$\begin{pmatrix} \begin{pmatrix} 0 \\ \frac{M}{r^2-2rM} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} \frac{M}{r^2-2rM} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ \begin{pmatrix} \frac{(r-2M)M}{r^3} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ -\frac{M}{r^2-2rM} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ -r+2M \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -(r-2M)\sin[\theta]^2 \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ \frac{1}{r} \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -\cos[\theta]\sin[\theta] \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{r} \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \cot[\theta] \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ \cot[\theta] \\ 0 \end{pmatrix} \end{pmatrix}$$

Difference

```
In[31]:= diffMatrix = Simplify[myGammaArray - truthArray]
```

```
Out[31]=
```

```
{ { {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0} },
  { {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0} },
  { {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0} },
  { {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0} } }
```

Should be all zero.

Recap from Stored Version

This derivation has been stored online. Run it from GitHub for a final check.

```
In[32]:= <<
```

```
"https://raw.githubusercontent.com/rebcabin/RelativityToolkit/main/compiler_poc.
wl"
```

» $g_{mn} \left\{ \left\{ -1 + \frac{2M}{r}, 0, 0, 0 \right\}, \left\{ 0, \frac{1}{1 - \frac{2M}{r}}, 0, 0 \right\}, \left\{ 0, 0, r^2, 0 \right\}, \left\{ 0, 0, 0, r^2 \sin[\theta]^2 \right\} \right\}$

» Γ from $\mathcal{UD}$

$$\begin{pmatrix} \begin{pmatrix} 0 \\ \frac{M}{r^2 - 2rM} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} \frac{M}{r^2 - 2rM} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ \begin{pmatrix} \frac{(r-2M)M}{r^3} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ -\frac{M}{r^2 - 2rM} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ -r + 2M \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -(r-2M)\sin[\theta]^2 \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ \frac{1}{r} \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -\cos[\theta]\sin[\theta] \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{r} \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \cot[\theta] \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ \cot[\theta] \\ 0 \end{pmatrix} \end{pmatrix}$$

» Γ from WFR

$$\begin{pmatrix} \begin{pmatrix} 0 \\ \frac{M}{r^2 - 2rM} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} \frac{M}{r^2 - 2rM} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ \begin{pmatrix} \frac{(r-2M)M}{r^3} \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ -\frac{M}{r^2 - 2rM} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ -r + 2M \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -(r-2M)\sin[\theta]^2 \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ \frac{1}{r} \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ -\cos[\theta]\sin[\theta] \end{pmatrix} \\ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{r} \end{pmatrix} & \begin{pmatrix} 0 \\ 0 \\ 0 \\ \cot[\theta] \end{pmatrix} & \begin{pmatrix} 0 \\ \frac{1}{r} \\ \cot[\theta] \\ 0 \end{pmatrix} \end{pmatrix}$$

» $\text{computed } \Gamma == \text{ground truth from WFR}$ True

Conclusion

In this notebook, $\mathcal{UD}$ automates one of the most error-prone exercises in differential geometry: calculating the Riemann curvature tensor from scratch. $\mathcal{UD}$'s automation avoids the eager-expansion trap of student lore: " Γ is not a tensor."

Via the gamma-glue mechanism, $\mathcal{UD}$'s covariant derivative puts Γ in a box and sends it inertly for later expansion in the ordinary product rule, ensuring correct derivative indices in comma notation.

The result is mathematically identical to the textbook definition, but is also structurally robust. Just

reconfigure $\mathcal{UD}$ for electromagnetism, Yang-Mills: any gauge theory.

As a bonus, we illustrate compiling $\mathcal{UD}$ notations onto `MetricTensor` and `ChristoffelSymbol` from the Wolfram Function Repository. This illustration is a proof-of-concept for a full compiler backend for $\mathcal{UD}$.

In future work, we will continue to extend and expand $\mathcal{UD}$ for more applications in relativity and gauge theory.

References

1. Beckman, Brian. *From the steam age to quantum fields: a unified approach via UD tensorial calculus*. Wolfram Community post.
2. Beckman, Brian. *General relativity in the UD calculus*. Wolfram Community post.
3. Beckman, Brian. *Formal Differential Geometry in the UD Calculus: Part 1*. Wolfram Community post.
4. Beckman, Brian. *Covariant Derivatives and Connections in the UD Calculus*. Wolfram Community post (to appear).
5. Wolfram Function Repository. *MetricTensor*. For component-based calculations.
6. Wolfram Function Repository. *ChristoffelSymbol*. For component-based calculations.
7. [MTW] Misner, Charles W.; Thorne, Kip S.; Wheeler, John Archibald. *Gravitation*. Princeton University Press, 2017.